

Simulation and Performance Analysis of a Direct-Mapped Instruction and Data Cache

Sudarshan Sudhakar
Roll No: CS23B2007

August 11, 2026

Abstract

This report describes the design and evaluation of a Python-based cache simulator that models a direct-mapped Instruction Cache (I-Cache) and Data Cache (D-Cache) for a system with a 64K-word main memory, a 2K-word cache, a 16-word block size, and 32-bit words/addresses. Three representative memory access patterns – sequential, repeated, and strided – are simulated for both instruction and data streams. For each run the simulator reports total accesses, cache lookups, hits, misses, hit rate, and miss rate, and further decomposes every miss into Compulsory (Cold), Conflict, and Capacity categories using the Hill-Smith three-C classification method. The results are used to discuss how spatial and temporal locality, together with the direct-mapped placement policy, govern cache efficiency.

Contents

1	System Specification	1
1.1	Derived Cache Geometry	1
2	Simulator Design	2
2.1	Direct-Mapped Cache Model	2
2.2	Classifying Misses into Cold / Conflict / Capacity	2
2.3	Access Patterns Simulated	3
3	Results	3
3.1	Sequential Access Pattern	3
4	Analysis and Discussion	4
4.1	Sequential Access (Table 3)	4
4.2	Repeated Access (Table 4)	4
4.3	Strided Access (Tables 5, 6)	5
4.4	Instruction Cache vs. Data Cache	5
4.5	Locality of Reference and Cache Mapping – Summary	5
5	Conclusion	6
A	Source Code	6

1 System Specification

Parameter	Value
Main memory size	64K words (2^{16} words)
Cache size (each of I-Cache, D-Cache)	2K words (2^{11} words)
Block size	16 words (2^4 words)
Word size	32 bits
Address size	32 bits
Mapping scheme	Direct Mapped

Table 1: Given system parameters

1.1 Derived Cache Geometry

Number of lines (blocks) in the cache:

$$\text{Lines} = \frac{\text{Cache size}}{\text{Block size}} = \frac{2048}{16} = 128 \text{ lines}$$

Since the cache is direct mapped, a 32-bit word address is split into three fields – *Tag*, *Index*, and *Offset*:

$$\text{Offset bits} = \log_2(16) = 4, \quad \text{Index bits} = \log_2(128) = 7, \quad \text{Tag bits} = 32 - 7 - 4 = 21$$

Field	Tag	Index	Offset
Width (bits)	21	7	4
Purpose	identifies which memory block	selects the 1-of-128 cache line	selects the word within a 16-word block

Table 2: Address decomposition used by the simulator (`cache.decompose`)

Assumption: the assignment specifies a single “2K-word cache” but asks that both an I-Cache and a D-Cache be modelled. We interpret this as a Harvard-style organisation in which the I-Cache and the D-Cache are two *independent* caches, each built to the given specification (2K words, direct mapped, 16-word blocks). They are simulated completely independently – an instruction access can never hit or evict a line in the D-Cache and vice versa.

2 Simulator Design

The simulator is written in Python 3 and organised into three modules:

- `cache.py` – the direct-mapped cache model, the address decomposition logic, and the three-C (Cold/Conflict/Capacity) miss classifier.
- `patterns.py` – generators that produce the address streams for the three required access patterns, for instructions and for data.
- `simulator.py` – the driver that runs every `pattern × {instruction, data}` combination through a fresh cache instance, prints a formatted report, and dumps the results to `results/results.json`.

2.1 Direct-Mapped Cache Model

Each cache is represented by two parallel arrays of length 128 (one entry per line): a **valid** bit and a **tag**. On every access the address is decomposed into (tag, index, offset); the **index** selects a single candidate line, and *exactly one* tag comparison (one “lookup”) is performed:

- If the line is valid and its stored tag matches $\Rightarrow$ **hit**.
- Otherwise $\Rightarrow$ **miss**; the block is loaded into that line (allocate-on-miss), overwriting whatever was previously resident there.

Because a direct-mapped cache checks only one line per access, the number of *cache lookups* reported below is always equal to the number of accesses (one tag comparison per access) – this is structurally different from a set-associative cache, where a single access can require several parallel tag comparisons.

2.2 Classifying Misses into Cold / Conflict / Capacity

We use the standard Hill–Smith “Three-C” method, implemented in **ThreeCClassifier**:

1. **Cold (Compulsory) miss** – the referenced memory block has never been accessed before in this run. Such a miss would occur even in an infinite cache and is unavoidable on first touch.
2. A reference **fully-associative LRU cache** of the *same capacity* (128 blocks) is run in parallel, purely as a measuring stick.
 - If the direct-mapped cache misses, the block is not cold, but the reference LRU cache *also* misses on it $\Rightarrow$ **Capacity miss** (the working set genuinely exceeds the cache’s total capacity, so no placement policy could have avoided this miss).
 - If the direct-mapped cache misses but the reference LRU cache *hits* $\Rightarrow$ **Conflict miss** (the miss is an artefact of direct-mapped placement – two blocks competing for the same index evicted each other, even though the overall working set would have fit).

Every miss produced by the direct-mapped cache is assigned to exactly one of these three buckets, and the simulator asserts $\text{Cold} + \text{Conflict} + \text{Capacity} = \text{Total misses}$ after every run as a self-check.

2.3 Access Patterns Simulated

All addresses are 32-bit *word* addresses into the 64K-word memory. Instruction streams are generated in the region starting at word 0, data streams in a disjoint region starting at word 0×8000 (32768), purely so the two are easy to reason about independently; the caches themselves are simulated completely separately regardless.

1. **Sequential (spatial locality)**. 4096 consecutive word addresses (256 cache blocks, i.e. $2 \times$ the cache’s 128-line capacity), modelling straight-line instruction execution or a linear array scan with no reuse.
2. **Repeated (temporal locality)**. Two sub-cases:
 - *Fits in cache*: a 32-word instruction loop body (2 blocks) executed 200 times, and a 16-word data window (1 block) reused 500 times – both comfortably smaller than the 128-line cache.

- *Exceeds cache capacity*: a 3072-word (192-block, $1.5\times$ capacity) instruction working set and a 4096-word (256-block, $2\times$ capacity) data working set, each swept 4 times, to show what happens when a “hot loop” is actually too large to stay resident.

3. **Strided**. Fixed-stride sweeps with stride $\in \{2, 4, 16\}$ words over 1024 accesses, showing spatial locality eroding as the stride grows (stride 16 = one full block, i.e. zero reuse within a block). A fourth, dedicated *conflict-demonstration* case round-robins across 4 addresses spaced exactly one cache-size (2048 words) apart, 256 times – these four blocks alias to the very same cache index, which isolates **conflict misses** from capacity effects (a fully-associative cache of equal size would need only 4 of its 128 lines to hold this entire working set).

3 Results

All figures below are the direct, un-edited output of `simulator.py` (see `results/results.json`).

3.1 Sequential Access Pattern

Cache / Case	Acc.	Lookups	Hits	Misses	Hit%	Miss%	Cold	Conflict	Capacity
I-Cache (sequential)	4096	4096	3840	256	93.75%	6.25%	256	0	0
D-Cache (sequential)	4096	4096	3840	256	93.75%	6.25%	256	0	0

Table 3: Sequential access pattern – I-Cache and D-Cache results

Cache / Case	Acc.	Lookups	Hits	Misses	Hit%	Miss%	Cold	Conflict	Capacity
I-Cache (working set fits in cache)	6400	6400	6398	2	99.97%	0.03%	2	0	0
D-Cache (working set fits in cache)	8000	8000	7999	1	99.99%	0.01%	1	0	0
I-Cache (working set > cache capacity)	12288	12288	11712	576	95.31%	4.69%	192	0	384
D-Cache (working set > cache capacity)	16384	16384	15360	1024	93.75%	6.25%	256	0	768

Table 4: Repeated (temporal locality) access pattern – I-Cache and D-Cache results

Cache / Case	Acc.	Lookups	Hits	Misses	Hit%	Miss%	Cold	Conflict	Capacity
I-Cache (stride = 2)	1024	1024	896	128	87.50%	12.50%	128	0	0
D-Cache (stride = 2)	1024	1024	896	128	87.50%	12.50%	128	0	0
I-Cache (stride = 4)	1024	1024	768	256	75.00%	25.00%	256	0	0
D-Cache (stride = 4)	1024	1024	768	256	75.00%	25.00%	256	0	0
I-Cache (stride = 16)	1024	1024	0	1024	0.00%	100.00%	1024	0	0
D-Cache (stride = 16)	1024	1024	0	1024	0.00%	100.00%	1024	0	0

Table 5: Strided access pattern (stride = 2, 4, 16 words) – I-Cache and D-Cache results

Cache / Case	Acc.	Lookups	Hits	Misses	Hit%	Miss%	Cold	Conflict	Capacity
I-Cache (index-aliasing conflict demo)	1024	1024	0	1024	0.00%	100.00%	4	1020	0
D-Cache (index-aliasing conflict demo)	1024	1024	0	1024	0.00%	100.00%	4	1020	0

Table 6: Strided access, conflict-demonstration case – 4 addresses exactly one cache-size (2048 words) apart, round-robin 256 times

Repeated Access Pattern

See Table 4 above.

Strided Access Pattern

See Tables 5 and 6 above.

4 Analysis and Discussion

4.1 Sequential Access (Table 3)

Both caches show an identical **93.75% hit rate**. With a 16-word block, every block delivers 1 miss followed by 15 hits as long as the stream never revisits an evicted block – exactly $256/4096 = 6.25\%$ miss rate. Every single miss is classified as **Cold**; there is no conflict or capacity component because each of the 256 blocks touched is accessed exactly once and then abandoned, so nothing is ever evicted and re-requested. This is the textbook signature of *spatial locality*: large blocks amortise the fixed cost of a miss over many subsequent hits, and the hit rate is entirely determined by the block size ($\frac{15}{16} = 93.75\%$), independent of cache size or associativity.

4.2 Repeated Access (Table 4)

Working set within capacity. With only 2 (I) or 1 (D) blocks being swept repeatedly, hit rates reach **99.97%** and **99.99%**: the entire working set is loaded once (a handful of cold misses) and then serviced from the cache for hundreds of iterations. This is the purest demonstration of *temporal locality* – exactly the loop / loop-carried-variable scenario the pattern is meant to model.

Working set exceeding capacity. Once the swept region grows past the cache’s 128-line capacity (192 blocks for I, 256 blocks for D), the hit rate falls to 95.31% / 93.75% and, critically, the miss breakdown now contains a large **Capacity** component (384 and 768 misses respectively) on top of the unavoidable cold misses – with *zero* conflict misses. Because the sweep revisits blocks in the same round-robin order every pass, a fully-associative LRU cache of equal size thrashes in exactly the same way as the direct-mapped cache, so every non-cold miss here is attributable purely to insufficient capacity, not to index collisions. This isolates the effect the assignment asks about: a loop or array that is individually “hot” but simply too large to fit will thrash regardless of how clever the placement policy is.

4.3 Strided Access (Tables 5, 6)

Increasing stride erodes spatial locality. For a plain strided sweep with no revisits, the hit rate falls monotonically as stride increases: $87.50\% \rightarrow 75.00\% \rightarrow 0.00\%$ for stride $2 \rightarrow 4 \rightarrow 16$. With a 16-word block, a stride of 2 still lands 8 of the 16 words in each block before moving to the next block (7 hits after the first miss on the block, so with count 1024 that is $\frac{1}{2} \cdot 1,024$ block-touches down to 128 unique blocks, each contributing 1 cold miss and 7 hits); stride 4 similarly halves the reuse again; and once the stride equals the block size (16), every single access lands in a brand-new block, spatial locality is completely destroyed, and the miss rate hits **100%**. As with the sequential pattern, every miss here is classified as Cold – nothing is ever revisited, so there is nothing for a conflict/capacity distinction to act on.

Isolating conflict misses. The dedicated conflict-demonstration case (Table 6) makes the cost of direct mapping explicit: only 4 blocks are ever touched (4 cold misses), and this tiny working set would trivially fit in *any* cache with ≥ 4 lines of *any* associativity. Yet because all 4 addresses are exactly one cache size (2048 words) apart, they map to the *same* index and

continuously evict one another. The result is a **100% miss rate** with 1020 of the 1024 misses classified as pure **Conflict** misses – the single largest source of avoidable misses observed in this study, and one that a capacity argument alone cannot explain (the reference fully-associative cache of equal total size hits almost every time). This is precisely the scenario direct-mapped caches are known to handle poorly, and is the reason set-associative caches exist.

4.4 Instruction Cache vs. Data Cache

Across every pattern, the I-Cache and D-Cache results are structurally identical in shape (same hit/miss ratios, same 3C breakdown), because both caches use the same geometry and are driven by address streams constructed with the same access pattern (only the base address differs). This is an expected artefact of testing the two caches with synthetic streams; in a real workload the I-Cache would additionally be shaped by branch behaviour (taken/not-taken changes sequential fetch into effectively strided/discontinuous access) while the D-Cache would be shaped by the specific data structures being manipulated (arrays → sequential/strided, scalars and loop counters → repeated). The simulator’s independent I/D modelling reflects a Harvard-style split cache, where instruction fetch pressure can never evict useful data, and vice versa – avoiding the interference that would occur in a shared unified cache.

4.5 Locality of Reference and Cache Mapping – Summary

- **Spatial locality** (exploited by block size) is the dominant factor for sequential and low-stride access: hit rate is essentially $1 - \frac{1}{\text{words per block used}}$, and degrades sharply once the access stride approaches or exceeds the block size.
- **Temporal locality** (exploited by cache capacity and residency) is the dominant factor for repeated/loop-style access, *provided the working set fits inside the cache*. When it doesn’t, capacity misses appear regardless of placement policy.
- **Cache mapping policy** (direct-mapped vs. associative) only becomes visible once addresses *alias* onto the same index. The conflict-demonstration experiment shows this can be catastrophic (100% miss rate) even for a working set that is trivially small relative to total cache capacity – the defining weakness of direct mapping relative to set-associative or fully-associative organisations.

5 Conclusion

The simulator confirms, with concrete measurements, the standard qualitative predictions of cache theory for a direct-mapped organisation: sequential access performance is governed by block size (spatial locality), repeated access performance is governed by whether the working set fits within cache capacity (temporal locality), and strided access exposes both a gradual loss of spatial locality as stride grows and, in the worst case (addresses aliasing to the same index), severe conflict misses that no amount of spare cache capacity can prevent. Because the I-Cache and D-Cache are modelled as independent direct-mapped structures, instruction and data traffic never interfere with one another, isolating each pattern’s effect cleanly. These results motivate the two classic remedies covered in a computer organisation course: larger/well-chosen block sizes to capture more spatial locality, and higher associativity (or software techniques such as array padding/blocking) to eliminate pathological conflict misses like the one demonstrated in Table 6.

A Source Code

The complete, runnable source (`cache.py`, `patterns.py`, `simulator.py`, `generate_report_tables.py`) is submitted alongside this report in the `src/` directory, and the raw JSON results in `results/results.json`. The core cache model and 3C classifier are reproduced below for reference.

Listing 1: `cache.py` – direct-mapped cache model and 3C classifier

```
1  """
2  Direct-Mapped Cache model + 3C (Compulsory / Conflict / Capacity) miss classifier.
3
4  System parameters (fixed by the assignment):
5      Main memory : 64K words (216 words)
6      Cache size  : 2K words (211 words)
7      Block size  : 16 words (24 words)
8      Word size   : 32 bits
9      Address size: 32 bits (word address -- byte offset within a word is ignored,
10                          i.e. addresses below refer to WORD numbers, not byte numbers)
11
12  Derived geometry:
13      Lines in cache = cache_size / block_size = 2048 / 16 = 128 lines
14      Offset bits    = log2(block_size)         = 4 bits
15      Index bits     = log2(num_lines)           = 7 bits
16      Tag bits       = addr_bits - index - off  = 32 - 7 - 4 = 21 bits
17  """
18
19  from dataclasses import dataclass, field
20
21
22  WORD_BITS = 32
23  MAIN_MEMORY_WORDS = 64 * 1024          # 64K words
24  CACHE_WORDS = 2 * 1024                 # 2K words
25  BLOCK_WORDS = 16                       # 16 words / block
26
27  NUM_LINES = CACHE_WORDS // BLOCK_WORDS # 128 lines
28  OFFSET_BITS = BLOCK_WORDS.bit_length() - 1 # 4
29  INDEX_BITS = NUM_LINES.bit_length() - 1 # 7
30  TAG_BITS = WORD_BITS - INDEX_BITS - OFFSET_BITS # 21
31
32
33  def decompose(addr: int):
34      """Split a 32-bit word address into (tag, index, offset)."""
35      offset = addr & (BLOCK_WORDS - 1)
36      index = (addr >> OFFSET_BITS) & (NUM_LINES - 1)
37      tag = addr >> (OFFSET_BITS + INDEX_BITS)
38      return tag, index, offset
39
40
41  def block_number(addr: int) -> int:
42      """The memory block (line-sized chunk of main memory) an address belongs to."""
43      return addr >> OFFSET_BITS
44
45
46  @dataclass
47  class MissBreakdown:
48      cold: int = 0
49      conflict: int = 0
50      capacity: int = 0
```

```

51
52     @property
53     def total(self):
54         return self.cold + self.conflict + self.capacity
55
56
57 @dataclass
58 class CacheStats:
59     name: str
60     accesses: int = 0
61     hits: int = 0
62     misses: int = 0
63     lookups: int = 0
64     miss_breakdown: MissBreakdown = field(default_factory=MissBreakdown)
65
66     @property
67     def hit_rate(self):
68         return self.hits / self.accesses if self.accesses else 0.0
69
70     @property
71     def miss_rate(self):
72         return self.misses / self.accesses if self.accesses else 0.0
73
74
75 class DirectMappedCache:
76     """A single direct-mapped cache (used independently for I-cache and D-cache)."""
77
78     def __init__(self, name: str):
79         self.name = name
80         self.valid = [False] * NUM_LINES
81         self.tag = [0] * NUM_LINES
82         self.stats = CacheStats(name=name)
83
84     def access(self, addr: int) -> bool:
85         """Perform one lookup. Returns True on hit, False on miss."""
86         tag, index, _ = decompose(addr)
87         self.stats.accesses += 1
88         self.stats.lookups += 1 # one tag comparison per direct-mapped access
89         if self.valid[index] and self.tag[index] == tag:
90             self.stats.hits += 1
91             return True
92         # miss -> load the block (allocate on miss)
93         self.valid[index] = True
94         self.tag[index] = tag
95         self.stats.misses += 1
96         return False
97
98
99 class FullyAssociativeLRUCache:
100     """
101     Reference model with the SAME capacity (in blocks) as the direct-mapped
102     cache, but fully associative with true LRU replacement. Used only to
103     separate capacity misses from conflict misses (Hill & Smith 3C method).
104     """
105
106     def __init__(self, capacity_blocks: int):
107         self.capacity = capacity_blocks

```



```

108     self._order = []          # list of block numbers, most-recently-used at end
109     self._set = set()
110
111     def access(self, blk: int) -> bool:
112         if blk in self._set:
113             self._order.remove(blk)
114             self._order.append(blk)
115             return True
116         # miss
117         if len(self._order) >= self.capacity:
118             evict = self._order.pop(0)
119             self._set.discard(evict)
120             self._order.append(blk)
121             self._set.add(blk)
122             return False
123
124
125 class ThreeCClassifier:
126     """
127     Classifies every miss of a direct-mapped cache access stream into
128     Cold (Compulsory) / Conflict / Capacity, following the standard
129     3C model:
130
131     - Cold miss      : the block has never been referenced before
132                       (would miss even in an infinite cache).
133     - Fully-assoc LRU cache of the SAME capacity is run in parallel:
134       * miss in fully-assoc LRU cache AND not cold -> Capacity miss
135       (would still miss even with full associativity, i.e. purely
136       because the working set exceeds cache capacity)
137     - Conflict miss : miss in the direct-mapped cache that is
138                       neither cold nor capacity, i.e. a fully
139                       associative cache of equal size would have hit
140                       (the miss is caused solely by index collisions).
141     """
142
143     def __init__(self, capacity_blocks: int):
144         self.seen_blocks = set()
145         self.lru_ref = FullyAssociativeLRUCache(capacity_blocks)
146         self.breakdown = MissBreakdown()
147
148     def classify(self, addr: int, dm_hit: bool):
149         blk = block_number(addr)
150         is_cold = blk not in self.seen_blocks
151         self.seen_blocks.add(blk)
152         lru_hit = self.lru_ref.access(blk)
153
154         if dm_hit:
155             return # no miss to classify
156
157         if is_cold:
158             self.breakdown.cold += 1
159         elif not lru_hit:
160             self.breakdown.capacity += 1
161         else:
162             self.breakdown.conflict += 1
163
164
165 def run_stream(name: str, addresses):

```

```
166     """Run an address stream through a fresh direct-mapped cache and
167     classify every miss into cold/conflict/capacity. Returns CacheStats."""
168     cache = DirectMappedCache(name)
169     classifier = ThreeCClassifier(NUM_LINES)
170
171     for addr in addresses:
172         hit = cache.access(addr)
173         classifier.classify(addr, hit)
174
175     cache.stats.miss_breakdown = classifier.breakdown
176     assert cache.stats.miss_breakdown.total == cache.stats.misses, (
177         f"3C classification mismatch for {name}: "
178         f"{cache.stats.miss_breakdown.total} != {cache.stats.misses}"
179     )
180     return cache.stats
```